AI-Powered CI/CD Failure Diagnosis Using RAG and LLM Agents

Author Name

***Abstract*—Continuous Integration and Continuous Delivery (CI/CD) pipelines are essential infrastructure in the modern software development lifecycle and diagnosis of failures is still a manual and time-consuming process. This paper introduces a prototype for an AI-driven diagnostic system using a Retrieval-Augmented Generation (RAG) approach with a Large Language Model (LLM) agent to automatically detect, diagnose and remediate frequent failures of CI/CD pipelines in a GitHub Actions environment. The system consumes failure logs, uses a vector store in ChromaDB to fetch contextually relevant historical incidents and documentation and generates structured root-cause analysis with remediation steps. We present the system architecture, methodology and a proposed evaluation matrix with 15-20 scenarios where failures are injected, such as dependency errors, Docker build failures, test regressions and Infrastructure-as-Code misconfigurations. This study fills a key gap in the field of intelligent DevOps automation, by providing a reproducible experimental framework and quantifiable performance metrics for LLM-powered pipeline diagnostics.**

***Index Terms*—CI/CD; GitHub Actions; Large Language Models; Retrieval-Augmented Generation; ReAct Agent; LangChain; DevOps Automation; ChromaDB; Failure Diagnosis; LangGraph**

I. Introduction

DevOps practices have become more widespread and the CI/CD pipelines are becoming a core software delivery solution. Development teams are known to be executing thousands of pipeline executions per day, yet pipeline failures remain a major source of productivity loss, according to industry surveys. Engineers invest a lot of time in manually checking logs, cross-referencing documents, and patching – all of which results in significant organisational time expenditure.

Even with the advances in monitoring and observability tooling, the cognitive effort needed to interpret failure logs and suggest remediations has been very manual. The DORA research programme in 2023 surveyed teams and discovered that the ones that dedicated more than two hours a week to diagnosing pipeline failures did not deploy as often and had higher rates of change failures, which showed the organisational cost of slow diagnosis (Forsgren, Smith and Humble, 2023). Given the advent of Large Language Models (LLMs) with impressive in-context reasoning capabilities and Retrieval-Augmented Generation (RAG) techniques, there is a strong opportunity to automate this diagnostic process to reduce the time between failure occurrence and resolution.

In this paper, an integrated prototype incorporating a vector store and semantic retrieval with ChromaDB, a Flask-based backend, a LangChain/LangGraph ReAct agent, and an LLM (OpenAI GPT or Anthropic Claude) is proposed for providing automated diagnostics for CI/CD failures. At inference time, the system is able to recall relevant past failures and documentation, so that the agent is able to reason within the actual operational context, not just based on parametric knowledge.

The main research question is: Does a ReAct based LLM agent with access to pipeline documentation and past failure logs, make accurate diagnoses and recommendations for remediations for common CI/CD pipeline failures in a GitHub Actions environment? This question is operationalised in a controlled experiment with injected failure scenarios, analyzed on diagnostic accuracy, relevance of suggested fixes and time-to-resolution.

II. State of the Art

The research on the intersection of AI and DevOps has significantly increased. The first sets of works concerned the detection of anomalies in streams of metrics with classical ML approaches. Recent work has investigated the use of neural language models for log analysis and prediction.

A. Log Analysis and Anomaly Detection

He et al. (2021) showed that the transformer-based models can achieve a higher clustering accuracy than the log parsing using traditional template-matching technique on large-scale log datasets. They proposed a LogBERT model which modified BERT's pre-training objectives to the log domain and acquired log-specific semantic representations. This work proved that the neural language understanding can be used for structured and semi structured Log data which was the key fundamental aspect for the proposed system.

B. Retrieval-Augmented Generation

Lewis et al. (2020) proposed the RAG framework, which showed that a combination of the parametric knowledge of LLM and the non-parametric knowledge from a dense passage retriever significantly boosted the performance on knowledge-intensive tasks. Later, RAG has been adopted for software engineering activities. In their study, Parvez et al. (2021) demonstrated that retrieval-augmented code generation can achieve better performance than generation-only models on various benchmarks, which justifies the use of RAG for CI/CD diagnostics where the past failure set acts as a "knowledge base" and can be retrieved.

C. LLM Agents for DevOps

Yao et al. (2023) proposed the ReAct (Reasoning + Acting) framework, which showed that small amounts of interleaving

chain-of-thought reasoning with tool-use actions can lead to significantly more reliable LLM performance in solving multi-step tasks than either reasoning or acting stand-alone. This paradigm can readily be applied to failure diagnosis, where the agent needs to reason about the contents of the log, get additional information about the context, and create a plan for remediation. Kang et al. (2024) further demonstrated the feasibility of using LLMs for automated bug reproduction, reinforcing the viability of LLM-supported software maintenance.

D. CI/CD-Specific Intelligence

Current commercial solutions such as GitHub Copilot for CLI and other AIOps platforms provide limited pipeline-specific intelligence, where they might alert on exceeding thresholds or search for keywords in logs, but do not semantically reason about the root cause. Although some academic research is performed around the diagnosis of failures in CI/CD, research on CI/CD failure diagnosis using RAG and agent frameworks remains limited, and the research gap is filled in this proposal. To the best of our knowledge, the system presented here is the first one to integrate a long-term vector store of past failure-fix pairs with a ReAct agent loop specifically evaluated on a handcrafted collection of GitHub Actions failure scenarios.

III. Research Question and Novelty

The main research questions are: What is the accuracy of the LLM agent with the ReAct prompt design to accurately diagnose and suggest remediations for common categories of pipeline failures based on the historical pipeline failure logs and GitHub Actions documentation via RAG approach?

Secondary hypotheses are: (H1) retrieval of semantically similar content from a curated content store (ChromaDB) will improve diagnostic accuracy compared to a no-retrieval baseline, (H2) there will be significant differences between the accuracy of diagnostic statements for different types of failures, with dependency mismatch errors more accurately diagnosed than ambiguous test failures, and (H3) at least 70% of failures will be included in a diagnostic statement by the agent within a single inference pass.

This research has three novelty aspects. The first one is that it specifically focuses on failure diagnosis for CI/CD, instead of software debugging, making it possible to build up a highly relevant domain knowledge base. Second, it also has persistent storage of historical failure-fix pairs, such that the ReAct agent framework can create a feedback loop, where failure fixes improve the retrieval of failures in the future. Third, it offers a community benchmark evaluation framework that you can reproduce: a set of injected scenarios of GitHub Actions failures and the ground truth remediations.

IV. Research Methodology

A. Research Context

This research is situated within intelligent DevOps automation and targets software engineering teams maintaining CI/CD pipelines using GitHub Actions. The prototype is developed and evaluated within a controlled academic setting at TU Dublin, simulating a realistic small-to-medium enterprise DevOps workflow. The study is timely because the rapid maturation of LLM reasoning capabilities and open-source agent frameworks (LangChain, LangGraph) now makes such a system feasible to build and evaluate. The findings are intended to be generalisable to other CI/CD platforms (GitLab CI, Jenkins) through the modular architecture of the prototype.

B. System Architecture

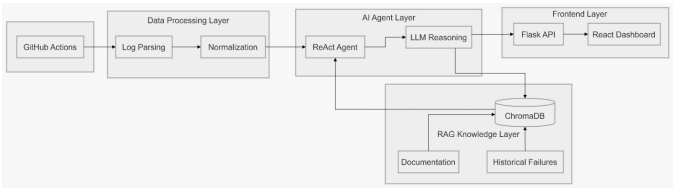

Fig. 1. System Architecture

The proposed system consists of a five-layered architecture. The CI/CD layer is built with GitHub Actions to provide a realistic set of logs for pipeline executions in various failure scenarios. The data processing layer performs the following operations: Log data is cleaned, parsed and normalised to structured failure data using Python's re module, pandas and json. The RAG layer uses sentence-transformers all-MiniLM-L6-v2 model to embed the processed logs and documentation, and stores them in a ChromaDB with on-disk persistent storage. The AI layer uses LangChain and LangGraph to implement a ReAct agent that calls the OpenAI or Claude API to do reasoning and generation. The front end layer offers a visualisation of the log to upload, diagnose display and retrieval visualisation via a React dashboard.

C. Knowledge Base Construction

The ChromaDB vector store contains some pre-populated documents: The official documentation of GitHub Actions; Docker troubleshooting guides; dependency resolution error patterns; YAML configuration issue patterns; IaC misconfiguration patterns; and historical failure patterns and fixes from our development process. The system chunks documents into segments of 512 tokens, with 64-token overlap, and embeds documents as dense vectors. During inference, grounding evidence is given to the agent by retrieving top-k=5 nearest neighbours using cosine similarity.

D. Agent Design

The ReAct agent has a similar observe–reason–act cycle. When it receives a failure log, it first parses the failure log to a structured log summary and then calls retrieval tools in ChromaDB to retrieve the context, reasons over the retrieved context to identify the category of failure, hypothesize on its root cause, evaluate the confidence score, and order remediation steps, producing a structured diagnosis. Once the diagnosis is successful, the failure-fix pair is again added to

the ChromaDB, thus continuously enhancing the knowledge base.

E. Experimental Design

The evaluation data set itself contains 15-20 scenarios of injected failures across four categories: (1) dependency errors (version conflicts, missing packages); (2) Docker build failures (failed to pull base image, invalidated layer caching); (3) test failures (flaky tests, assertion failures depending on the environment); and (4) IaC misconfigurations (invalid Terraform HCL, schema violations in the Kubernetes manifests). Each scenario is set up by purposefully adding a certain type of fault to a passing pipeline and saving the GitHub Actions log.

The system is tested under three different setups: (a) LLM Only (no retrieval); (b) RAG with documentation only; and (c) RAG with documentation and historical failure logs. Diagnostic accuracy (is the root cause correctly identified), remediation actionability (does the suggested remediation fix the failure when applied), and mean time-to-diagnosis, compared to manual diagnosis by an expert, are examples of evaluation metrics. Two independent evaluators evaluate each scenario using a standardised rubric and inter-rater agreement is determined by Cohen's kappa to guarantee that the evaluation is reliable. Further, the passages retrieved by the agent are analysed for relevance in terms of the Normalised Discounted Cumulative Gain (NDCG), a standard information-retrieval metric that measures how well the most relevant documents are ranked at the top of results, which allows to quantify retrieval quality without taking the accuracy of the final diagnosis into account.

F. Ethical and Practical Considerations

All failure logs used in the evaluation are synthetically generated in isolated GitHub Actions environments, no production logs or personal data are involved. As the study does not involve human participants, no ethics board approval is required under TU Dublin's research ethics guidelines. API usage for LLM inference is rate-limited and token-budgeted to ensure experimental reproducibility and control costs. All synthetic data and evaluation scripts will be stored in a version controlled repository. The system provides AI generated diagnostic recommendations rather than final solutions, ensuring engineers retain full control over remediation actions on live infrastructure.

V. Expected Results and Conclusions

The results of the research will show that the RAG augmented agent significantly improves the performance of the "no retrieval" baseline on diagnosing accuracy, especially for failure categories that are well represented in the knowledge base, like dependency errors and Docker failures. The historical-failure RAG condition is predicted to outperform the documentation-only condition, thus supporting the value of building up operational incident knowledge. Quantitatively, we expect to have a higher diagnostic accuracy than 75% for the full RAG condition for all failure categories and estimate the diagnostic accuracy for the LLM-only baseline to be in the range of 45-55%, which gives a strong empirical basis for the retrieval-augmentation in this space. Results will be presented as per-category accuracy tables comparing the three ablation conditions, box plots of time-to diagnosis distributions, a confusion matrix of failure-category classifications, NDCG scores for retrieval quality as well as a Cohen's kappa inter-rater reliability report. Statistical significance will be assessed using paired t-tests or Wilcoxon signed-rank tests where appropriate.

Results are also likely to show that current LLM agents get errors for highly context dependent tasks like finding flaky tests, where it takes deep understanding of the codebase beyond log context to tell if it is a real regression or not. These results will be used to make concrete suggestions on how to build the knowledge base, how to retrieve the information and how to design the agents tools for production DevOps settings. When the agent does not generate a proper diagnosis, the error analysis will classify failure modes as retrieval misses, reasoning errors and hallucinated remediations and give specific directions for future improvement.

The main contribution is a prototype and evaluation matrix which can be replicated by the research community as a starting point for further research on intelligent CI/CD automation. A dataset of failure scenarios to be injected, scripts for evaluation and system configuration will be publicly available. This is part of a larger effort to leverage LLM intelligence for software operations, minimizing downtime and freeing up engineering time for more complex activities.

VI. Summary

In this paper, a research idea to design an AI-based CI/CD failure diagnosis system using a RAG architecture and an LLM agent with ReAct has been proposed. The state-of-the-art review shows that there is a clear gap in the area of applying these techniques for GitHub Actions pipeline diagnostics in a domain specific way. The proposed methodology includes three ablation conditions and is controlled, reproducible and measurable by evaluation criteria. The expected outcomes aim to provide valuable practical insights and experimental data on the effectiveness of retrieval-augmented LLM agents in DevOps automation scenarios. Future studies will focus on making the system work in multi-pipeline use cases and addressing the integration of issue tracking systems to automatically create tickets for incidents, as well as exploring options for fine-tuning on domain-specific log corpora to further decrease the reliance of the system on general-purpose LLM capabilities.

References

[1] Forsgren, N., Smith, D. and Humble, J. (2023) 2023 State of DevOps Report. Google Cloud / DORA. Available at: https://dora.dev/research/ (Accessed: 10 May 2026).

[2] He, W., Zhu, L., Zheng, J., Lyu, M.R. and King, I. (2021) 'LogBERT: Log Anomaly Detection via BERT', in Proceedings of the International Joint Conference on Neural Networks (IJCNN), pp. 1–8. Available at: https://arxiv.org/abs/2103.04475 (Accessed: 10 May 2026).

[3] Kang, S., Yoon, J., Askarbekkyzy, N. and Yoo, S. (2024) ‘Evaluating diverse large language models for automatic and general bug reproduction’, IEEE Transactions on Software Engineering, 50(10), pp. 2677–2694. Available at: https://arxiv.org/abs/2311.04532 (Accessed: 10 May 2026).

[4] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rockt ¨ aschel, T., Riedel, S. and Kiela, D. ¨ (2020) ‘Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks’, in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474. Available at: https://arxiv.org/abs/2005.11401 (Accessed: 10 May 2026).

[5] Parvez, M.R., Ahmad, W.U., Chakraborty, S., Ray, B. and Chang, K.W. (2021) ‘Retrieval Augmented Code Generation and Summarization’, in Findings of the Association for Computational Linguistics: EMNLP 2021, pp. 2719–2734. Available at: https://arxiv.org/abs/2108.11601 (Accessed: 10 May 2026).

[6] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023) ‘ReAct: Synergizing Reasoning and Acting in Language Models’, in Proceedings of the International Conference on Learning Representations (ICLR). Available at: https://arxiv.org/abs/2210.03629 (Accessed: 10 May 2026).